

Guía de Programación

Este documento aspira a combinar toda la información necesaria para empezar a programar el programa de administración del experimento de microscopía.

Requerimientos:

Para desarrollar este proyecto es necesario

- Un conocimiento general de Python
- Conocimiento de librería Lantz¹
- Conocimiento de librería QT
- Conocimiento de librerías de drivers²

1: Hay una guía de Lantz en esta carpeta de documentación.

2: Los manuales de los driver se pueden encontrar en esta carpeta

Estructura de los Archivos:

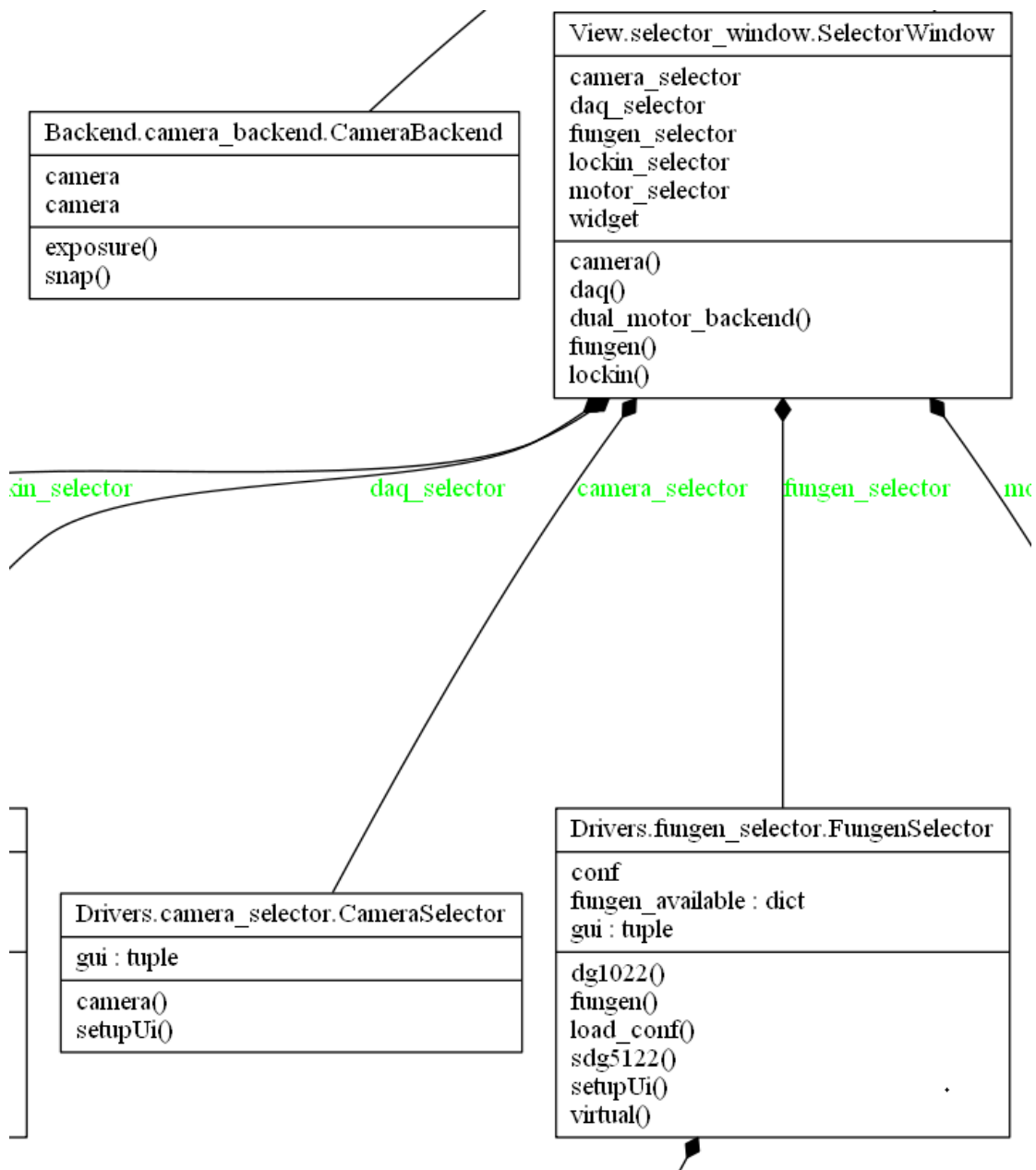
Los Archivos están distribuidos en las carpetas por su funcionalidad de la siguiente manera:

- Assets: Esta carpeta contiene los archivos de imágenes y otros necesarios para funcionar el programa que no son código. Ej.: el icono de la aplicación.
- Backend: Esta carpeta contiene los archivos de Python que se refieren a la interacción con los drivers y el código que administra la funcionalidad más compleja que se muestra en la interfaz de usuarios. Ej.: la creación de operaciones en la pantalla de selección de operaciones.
- Documentation: Esta carpeta contiene todos los archivos que explican la funcionalidad del programa, como instalarlo y como desarrollar para el. Ej.: Esta guía.
- Driver: Esta carpeta contiene todos los archivos de código relacionados a la selección, configuración y administración de los dispositivos de medición que utiliza el programa. Internamente tiene una subcarpeta para cada dispositivo puntual con el cual interactúa el programa. Ej.: El driver de cámara virtual.
- Model: Esta carpeta contiene todos los archivos de código que refieren a definir e representar los datos subyacentes que maneja el programa. Ej. Los puntos de medición.
- Results: Esta carpeta contiene los resultados de las corridas de medición.
- UnitTest: Esta carpeta contiene los archivos de código relacionados con el testeo de secciones unitarias de código. Ej.: Test de la creación de la escala de colores para el mapa
- View: Esta carpeta contienen los archivos de código relacionados a la administración de las ventanas del programa y las funciones visuales. En una subcarpeta llamada frontend guardo las diferentes secciones visuales de la pantalla y la administración de los botones y controles que se ven en la pantalla. Y como subcarpeta ahí se encuentran los archivos del qt designer .ui que indican como se ve la pantalla gráficamente. Ej. Program_window.py administra como se ve la pantalla principal.

Diagrama UML de las clases:

Se puede encontrar un diagrama uml de todos los archivos: Clases de Microscopia.png

Ej. de una sección



Referencia

La referencia es una lista de todos los módulos y sus respectivas clases y funciones presentes en su interior. La referencia se encuentra en el archivo web [“Documentation Reference.html”](#). Puede ver mas de como se genera en su propia guía.

Documentation Reference

“

Name

-  [Backend.camera_backend.html](#)
-  [Backend.experiment_backend.html](#)
-  [Backend.focus_backend.html](#)
-  [Backend.frequency_backend.html](#)

Ejemplo del módulo Model.point:

Model.point

Classes

[builtins.object](#)
[Point](#)

class **Point**([builtins.object](#))

[Point](#)(x: float, y: float, freq: int, n=1, display_x: int = 0, display_y: int = 0)

Class that represents a point to be measured in the round

Methods defined here:

```
__init__(self, x: float, y: float, freq: int, n=1, display_x: int = 0, display_y: int = 0)
    :param x: X coordinate in the camera that represented this point
    :param y: Y coordinate in the camera that represented this point
    :param freq: Frequeency at wich the point should be measured
    :param n: Amount of times the point should be re read to average the results
    :param display_x: X coordinate where the point should be represented in a map of the operation
    :param display_y: Y coordinate where the point should be represented in a map of the operation
```

```
str (self)
```

Pueden verse ahí los comentarios del programa y ver que funciones y clases están definidas ahí.